



System Design

CHAPTER 2: Back of the envelope Estimation

Dr. Farid Feyzi

1404

BACK-OF-THE-ENVELOPE ESTIMATION

- sometimes you are asked to estimate system capacity or performance requirements using a back-of-the-envelope estimation. According to Jeff Dean, Google Senior Fellow, "back-of-the-envelope calculations are estimates you create using a combination of thought experiments and common performance numbers to get a good feel for which designs will meet your requirements"
- You need to have a good sense of scalability basics to effectively carry out back-of-the-envelope estimation.
- The following concepts should be well understood: power of two [2], latency numbers every programmer should know, and availability numbers.

Power of two

- Power of two although data volume can become enormous when dealing with distributed systems, calculation all boils down to the basics.
- To obtain correct calculations, it is critical to know the data volume unit using the power of 2. A byte is a sequence of 8 bits.
- An ASCII character uses one byte of memory (8 bits). Below is a table explaining the data volume unit (Table 2-1).

Power	Approximate value	Full name	Short name
10	1 Thousand	1 Kilobyte	1 KB
20	1 Million	1 Megabyte	1 MB
30	1 Billion	1 Gigabyte	1 GB
40	1 Trillion	1 Terabyte	1 TB
50	1 Quadrillion	1 Petabyte	1 PB

Latency numbers every programmer should know

- Dr. Dean from Google reveals the length of typical computer operations in 2010 [1].
- Some numbers are outdated as computers become faster and more powerful. However, those numbers should still be able to give us an idea of the fastness and slowness of different computer operations.

Operation name	Time
L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	100 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	10,000 ns = 10 μ s
Send 2K bytes over 1 Gbps network	20,000 ns = 20 μ s
Read 1 MB sequentially from memory	250,000 ns = 250 μ s
Round trip within the same datacenter	500,000 ns = 500 μ s
Disk seek	10,000,000 ns = 10 ms
Read 1 MB sequentially from the network	10,000,000 ns = 10 ms
Read 1 MB sequentially from disk	30,000,000 ns = 30 ms
Send packet CA (California) ->Netherlands->CA	150,000,000 ns = 150 ms

Latency numbers every programmer should know

- A Google software engineer built a tool to visualize Dr. Dean's numbers.
- The tool also takes the time factor into consideration.
- Figure 2-1 shows the visualized latency numbers as of 2020 (source of figures: reference material [3]).

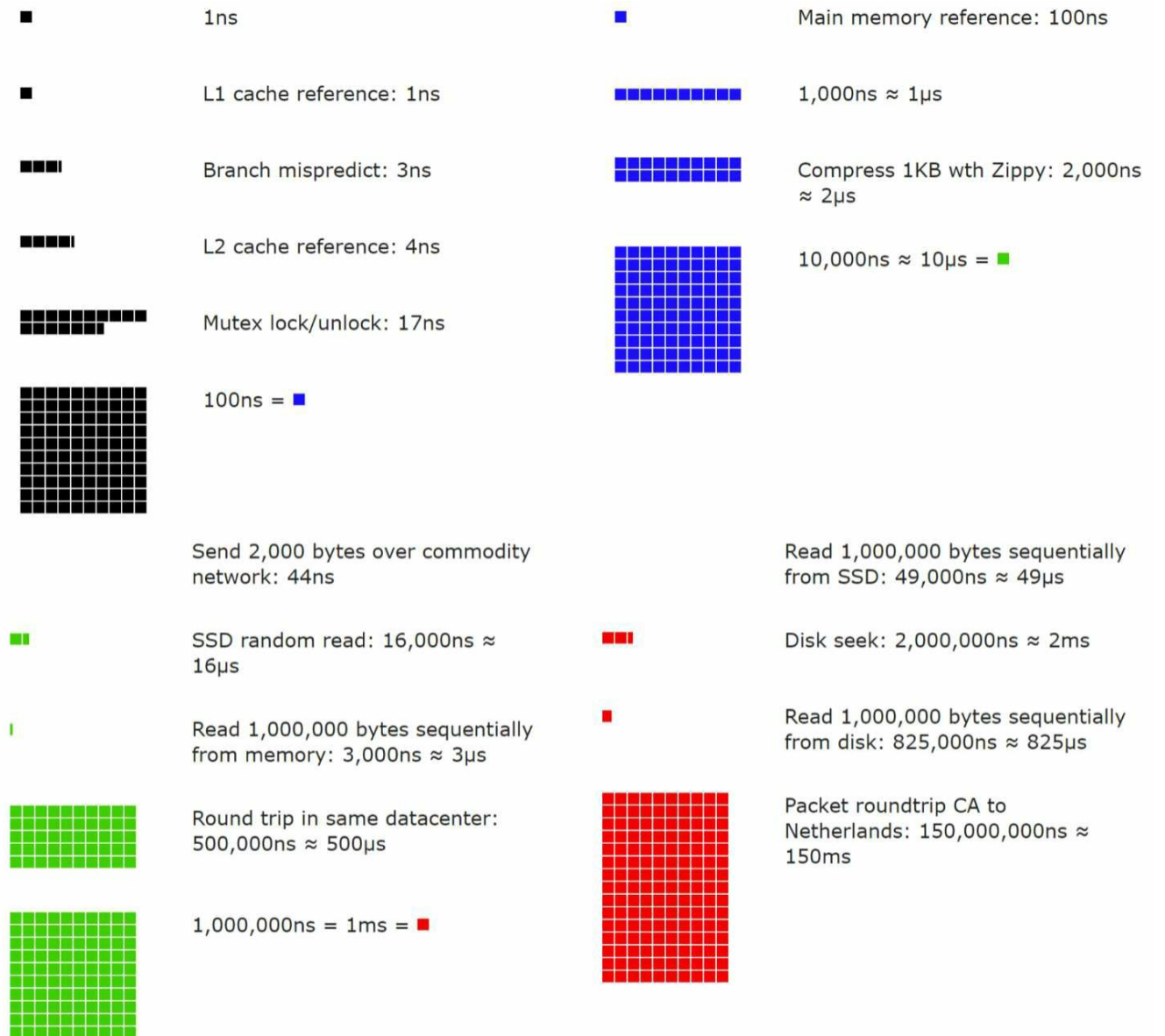


Figure 2-1

Latency numbers every programmer should know

- By analyzing the numbers in Figure 2-1, we get the following conclusions:
 - Memory is fast but the disk is slow.
 - Avoid disk seeks if possible.
 - Simple compression algorithms are fast.
 - Compress data before sending it over the internet if possible.
 - Data centers are usually in different regions, and it takes time to send data between them.

Availability numbers

- High availability is the ability of a system to be continuously operational for a desirably long period of time.
- High availability is measured as a percentage, with 100% means a service that has 0 downtime. Most services fall between 99% and 100%.
- A service level agreement (SLA) is a commonly used term for service providers.
- This is an agreement between you (the service provider) and your customer, and this agreement formally defines the level of uptime your service will deliver.
- Cloud providers Amazon [4], Google [5] and Microsoft [6] set their SLAs at 99.9% or above.
- Uptime is traditionally measured in nines. The more the nines, the better. As shown in Table 2-3, the number of nines correlate to the expected system downtime.

Availability numbers

Availability %	Downtime per day	Downtime per week	Downtime per month	Downtime per year
99%	14.40 minutes	1.68 hours	7.31 hours	3.65 days
99.99%	8.64 seconds	1.01 minutes	4.38 minutes	52.60 minutes
99.999%	864.00	6.05 seconds	26.30 seconds	5.26 minutes
99.9999%	86.40 milliseconds	604.80	2.63 seconds	31.56 seconds

Table 3

Example: Estimate Twitter QPS and storage requirements

- Let's say we have 500M daily active users. If each user performs an average of 2 tweets per day (most users are lurkers, some are power users), that gives us 1 billion tweets created daily.
- **Write QPS for tweets:** $1 \text{ billion tweets} / 86,400 \text{ seconds} = \sim 11,500 \text{ tweets/second}$ on average Peak could be 3x average = $\sim 35,000 \text{ tweets/second}$
- **Read QPS:** If our read/write ratio is 1000:1, we're looking at: 11.5M reads/second on average 35M reads/second at peak

Example: Estimate Twitter QPS and storage requirements

- Let's think about timeline refreshes instead. If each user refreshes their timeline 20 times per day: $500\text{M users} \times 20 \text{ refreshes} = 10 \text{ billion timeline loads daily}$ $10\text{B} / 86,400 = \sim 115,000 \text{ timeline loads/second average}$
- Peak: $\sim 350,000 \text{ timeline loads/second}$
- Storage requirements: Each tweet: $\sim 280 \text{ bytes text} + \sim 500 \text{ bytes metadata} = \sim 800 \text{ bytes}$ Media attachment: $\sim 200\text{KB average (mix of images/videos)}$
- If 20% of tweets have media: $1\text{B tweets} \times 0.2 \times 200\text{KB} = 40\text{TB of media daily}$
- Text storage: $1\text{B tweets} \times 800 \text{ bytes} = 800\text{GB daily}$ Over 3 years:
- Text: $\sim 1\text{PB}$
- Media: $\sim 45\text{PB}$

Example: Estimate Twitter QPS and storage requirements

- Memory requirements for caching: If we cache the most recent 20% of tweets (hot data): $200\text{M tweets} \times 800 \text{ bytes} = 160\text{GB}$ for tweet cache For timeline caches (assuming we pre-compute for active users): $100\text{M active users} \times 50\text{KB timeline} = 5\text{TB}$ of timeline cache